

How a Perceptron Learns

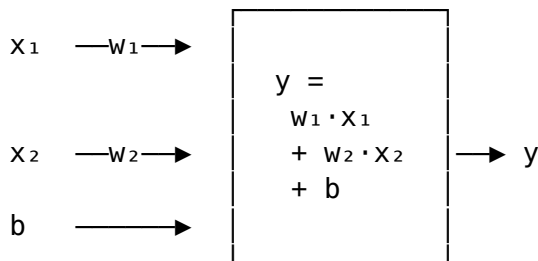
Forward and Backward, Atom by Atom

This document walks through every single arithmetic operation that happens when a simple two-input perceptron makes one prediction and learns one step from it. By the end you will have traced — **by hand** — exactly what `loss.backward()` does inside the `Value` class. The same algorithm that trains GPT, just shrunk down so every number fits on the page.

Companion to: the Tiny GPT 16-point walkthrough. This is the “atomic” version of what the `Value` class does. Once you can do these calculations by hand on paper, the autograd code reads like English.

1. What is a perceptron?

A perceptron is the simplest possible “neural network”: one neuron, a handful of inputs, and a single output. It computes a weighted sum of its inputs plus a bias term:



The weights w_1 , w_2 , and the bias b are **learned**. The inputs x_1 , x_2 are **given**. The output y is the prediction.

When training, we compare y to the correct answer (the **target**) and adjust the weights so y gets closer to it next time. That is exactly what we’re going to do — step by step.

2. The example we’ll trace

We need concrete numbers so every step is checkable.

Inputs and target (given to us, not learned):

Quantity	Value
x_1	2.0
x_2	3.0
target	4.0

Current weights (chosen randomly at the start of training):

Quantity	Value
w_1	1.0
w_2	0.5
b	-0.5

The loss we'll use is **squared error** between the prediction and the target. In Python:

```
m1 = w1 * x1
m2 = w2 * x2
s  = m1 + m2
z  = s + b
e  = z - target    # the "error"
L  = e ** 2        # squared error
```

Six lines. Each one creates a `Value` object that knows its own **data**, **children** (which inputs produced it), and **local derivatives** (one per child). Six `Value` nodes total, plus six leaf nodes for the inputs and constants.

3. Forward pass — six new `Value` nodes

We go line by line. For each operation we record three things: the forward value, the children, and the local derivatives.

Step	Code	data	children	local derivatives
1	<code>m1 = w1 * x1</code>	$1.0 \times 2.0 = \mathbf{2.0}$	<code>(w1, x1)</code>	<code>(x1.data, w1.data) = (2.0, 1.0)</code>
2	<code>m2 = w2 * x2</code>	$0.5 \times 3.0 = \mathbf{1.5}$	<code>(w2, x2)</code>	<code>(x2.data, w2.data) = (3.0, 0.5)</code>
3	<code>s = m1 + m2</code>	$2.0 + 1.5 = \mathbf{3.5}$	<code>(m1, m2)</code>	<code>(1, 1)</code>
4	<code>z = s + b</code>	$3.5 + (-0.5) = \mathbf{3.0}$	<code>(s, b)</code>	<code>(1, 1)</code>
5	<code>e = z - target</code>	$3.0 - 4.0 = \mathbf{-1.0}$	<code>(z, target)</code>	<code>(1, -1)†</code>
6	<code>L = e ** 2</code>	$(-1.0)^2 = \mathbf{1.0}$	<code>(e,)</code>	<code>(2·e.data,) = (-2.0,)‡</code>

† The actual `Value` class implements $a - b$ as $a + (-b)$, with local grads $(1, 1)$ on $(a, -b)$. The end-result gradient is the same; we use the simpler $(1, -1)$ form for readability.

‡ The power rule says $d(e^n)/de = n \cdot e^{n-1}$. With $n = 2$, that's $2 \cdot e$.

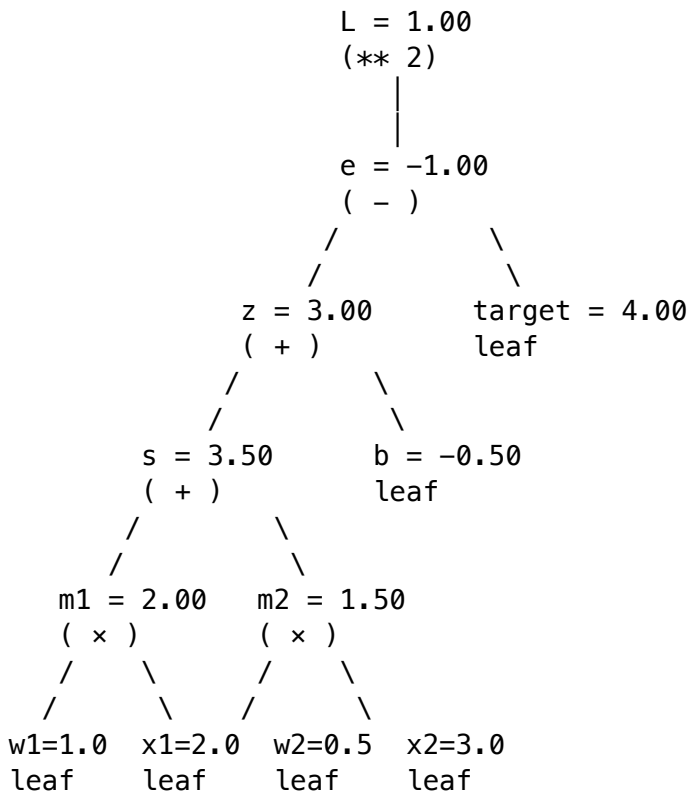
Notice the only “calculus” anywhere in this table is in the **local derivatives** column — three rules, each looked up from a textbook once:

- Product rule: $d(a \cdot b)/da = b$, $d(a \cdot b)/db = a$
- Sum rule: $d(a + b)/da = 1$, $d(a + b)/db = 1$
- Power rule: $d(e^n)/de = n \cdot e^{n-1}$

That's it. The machine doesn't need to know calculus — we taught it three rules.

4. The computation graph after the forward pass

After those six lines, the graph looks like this. Every node has its **data** (forward value) filled in. The shape on the right of each node tells you which operator created it.



Every leaf is something we either chose (w_1 , w_2 , b) or were given (x_1 , x_2 , $target$). Every internal node was created by an operator. The whole graph was built bottom-up by the six lines of Python.

The prediction the perceptron made is the data on node z : **3.00**. The correct answer was **4.0**. The squared error sitting on L is **1.0**. Now we need to nudge w_1 , w_2 , b so that next time the prediction is closer to 4.0.

5. Backward pass — chain rule, one edge at a time

We call `L.backward()`. This walks the graph in **reverse topological order** — parents before children — and applies the chain rule.

The rule is the same at every step. For each (`child`, `local`) pair on the node we're visiting:

```
child.grad += local * v.grad
```

That single line, repeated, is the entire backward pass.

Initialise: `L.grad = 1.0`. (The derivative of L with respect to itself is 1.)

Now walk:

#	Visit	Pushes grad into	Calculation	Result
1	L (grad=1.0)	e	local = 2·e.data = -2.0; e.grad += -2.0×1.0	e.grad = -2.0
2	e (grad=-2.0)	z target	local = +1; z.grad += 1×-2.0 local = -1; target.grad += -1×-2.0	z.grad = -2.0 target.grad = +2.0
3	z (grad=-2.0)	s	local = 1; s.grad += 1×-2.0	s.grad = -2.0
4	s (grad=-2.0)	b	local = 1; b.grad += 1×-2.0	b.grad = -2.0
		m1	local = 1; m1.grad += 1×-2.0	m1.grad = -2.0
		m2	local = 1; m2.grad += 1×-2.0	m2.grad = -2.0
5	m1 (grad=-2.0)	w1	local = x1.data = 2.0; w1.grad += 2.0×-2.0	w1.grad = -4.0
		x1	local = w1.data = 1.0; x1.grad += 1.0×-2.0	x1.grad = -2.0
6	m2 (grad=-2.0)	w2	local = x2.data = 3.0; w2.grad += 3.0×-2.0	w2.grad = -6.0
		x2	local = w2.data = 0.5; x2.grad += 0.5×-2.0	x2.grad = -1.0

After the walk, every leaf has its derivative with respect to the loss:

Leaf	grad
w1	-4.0
w2	-6.0
b	-2.0
x1	-2.0
x2	-1.0
target	+2.0

The three in bold are what we actually wanted. The others we'll discard — x1 and x2 are inputs we don't control, and target is the ground truth.

6. Sanity check: differentiate by hand

The autograd's answer for w1.grad was **-4.0**. Let's verify with a direct partial derivative.

The loss as a single formula:

$$L = (w1 \cdot x1 + w2 \cdot x2 + b - \text{target})^2$$

Differentiating with respect to w1 using the chain rule directly:

$$\begin{aligned}
 dL/dw1 &= 2 \cdot (w1 \cdot x1 + w2 \cdot x2 + b - \text{target}) \cdot x1 \\
 &= 2 \cdot (-1.0) \cdot 2.0 \\
 &= -4.0 \quad \checkmark
 \end{aligned}$$

Same for w2:

$$dL/dw2 = 2 \cdot (-1.0) \cdot 3.0 = -6.0 \quad \checkmark$$

And b:

$$dL/db = 2 \cdot (-1.0) \cdot 1 = -2.0 \quad \checkmark$$

Every gradient the autograd produced matches the by-hand differentiation. **They are the same calculation.** Autograd just automates the bookkeeping so we never have to write the chain rule out by hand for a 10,000-step expression.

7. The Python code that did this for us

Here is the actual Value class (trimmed for clarity). Compare it to what we did by hand above.

```
class Value:
    def __init__(self, data, children=(), local_grads=()):
        self.data = data
        self.grad = 0.0
        self._children = children
        self._local_grads = local_grads

    def __mul__(self, other):
        # Product rule.
        return Value(self.data * other.data,
                      (self, other),
                      (other.data, self.data))

    def __add__(self, other):
        # Sum rule.
        return Value(self.data + other.data,
                      (self, other),
                      (1.0, 1.0))

    def __pow__(self, n):
        # Power rule.
        return Value(self.data ** n,
                      (self,),
                      (n * self.data ** (n - 1),))

    def backward(self):
        topo, visited = [], set()
        def build_topo(v):
            if id(v) in visited: return
            visited.add(id(v))
            for child in v._children:
                build_topo(child)
            topo.append(v)
        build_topo(self)
```

```

self.grad = 1.0
for v in reversed(topo):
    for child, local in zip(v._children, v._local_grads):
        child.grad += local * v.grad        # ← the chain rule

```

The line at the very bottom is the entire computation we did by hand in §5:

```
child.grad += local * v.grad
```

That single line, executed 10 times in the right order, did everything in §5's table.

8. One step of learning

Now that we know the gradients, the optimiser nudges each weight against its gradient:

```

w1.data -= learning_rate * w1.grad
w2.data -= learning_rate * w2.grad
b.data  -= learning_rate * b.grad

```

With `learning_rate = 0.01`:

Parameter	Old value	Update	New value
w1	1.0	$-0.01 \times (-4.0)$	1.04
w2	0.5	$-0.01 \times (-6.0)$	0.56
b	-0.5	$-0.01 \times (-2.0)$	-0.48

Re-run the forward pass with the updated weights:

```

z_new = 1.04 · 2.0 + 0.56 · 3.0 + (-0.48)
       = 2.08 + 1.68 - 0.48
       = 3.28

```

```
L_new = (3.28 - 4.0)² = (-0.72)² = 0.5184
```

The loss dropped from **1.0** to **0.5184**. The perceptron has learned — a little. Repeat this loop for a few thousand documents and the loss approaches zero.

That is **training**. Forward to make a prediction. Backward to find out who is wrong and by how much. Step to nudge the weights. Repeat.

9. The big picture

Scale comparison:

	Perceptron (this doc)	GPT (microgpt-edu)
Trainable weights	3	4,192
Internal Value nodes per step	6	~10,000
Backward chain-rule applications	10	~20,000

	Perceptron (this doc)	GPT (microgpt-edu)
Training steps	~1,000	1,000
Pure-Python compute per step	microseconds	~50 milliseconds

The autograd code in `Value` is **identical** between the two. Same class, same `backward()` method, same chain-rule line. Nothing changes between training a perceptron and training GPT except how many nodes end up in the graph.

That is the most important takeaway from this walkthrough. The complexity of a neural network does **not** live in the autograd engine. It lives in **how many operations you string together to define the model**. Autograd is mechanical. Once you have it, you can write any model you like and gradients come for free.

Recap in one sentence

A perceptron, a transformer, and every neural network in between share the same training loop: **build a `Value` graph forward, propagate 1.0 through it backward via `child.grad += local * v.grad`, then nudge each weight against its gradient. Repeat.**